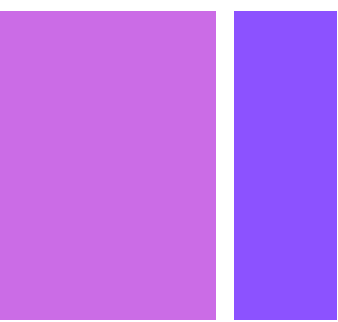
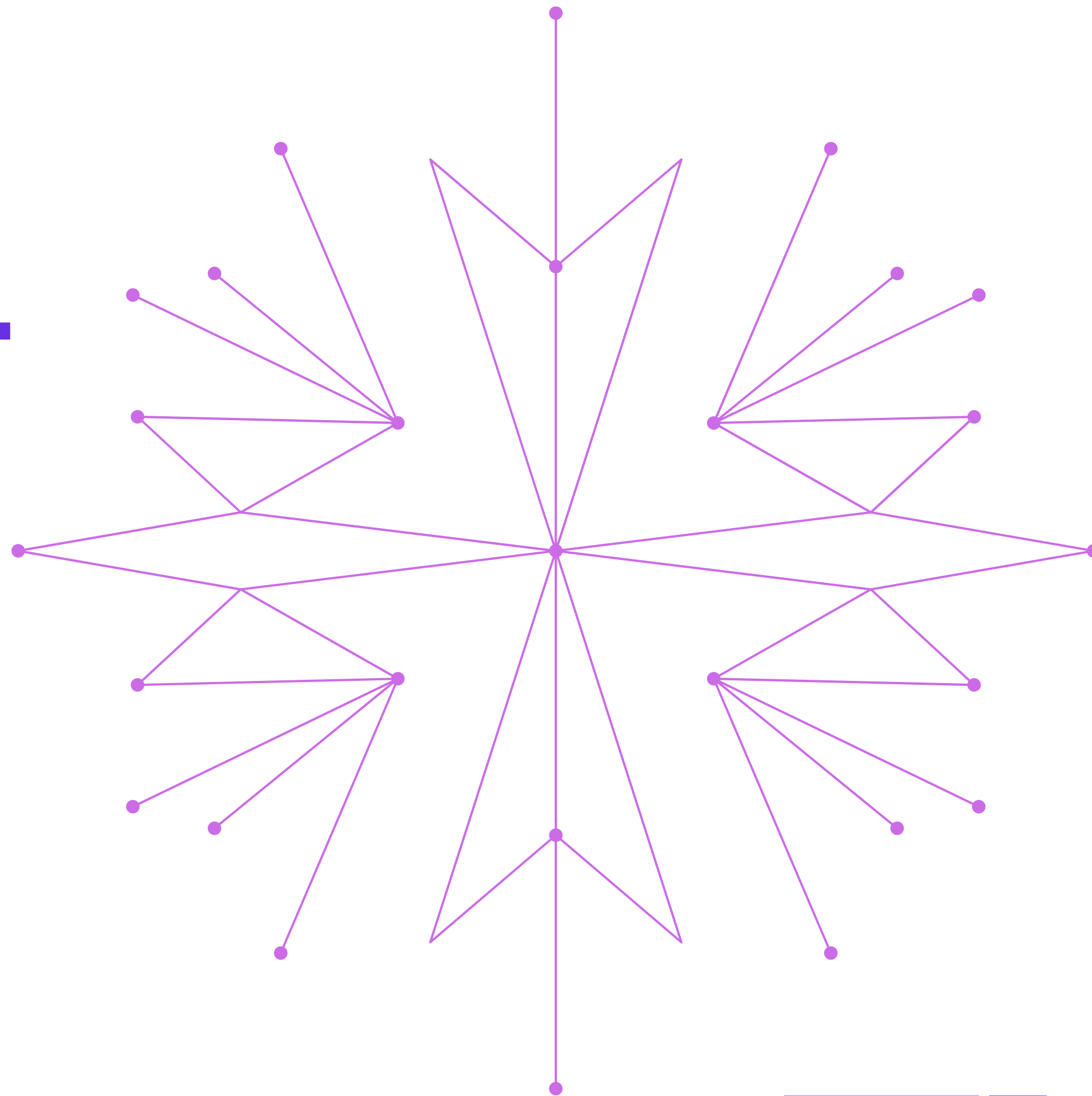
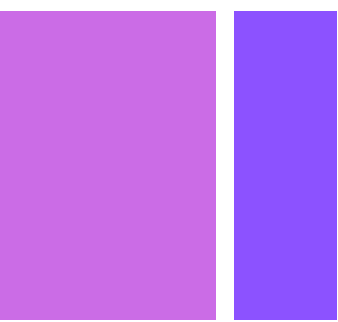




WELCOME TO HELL

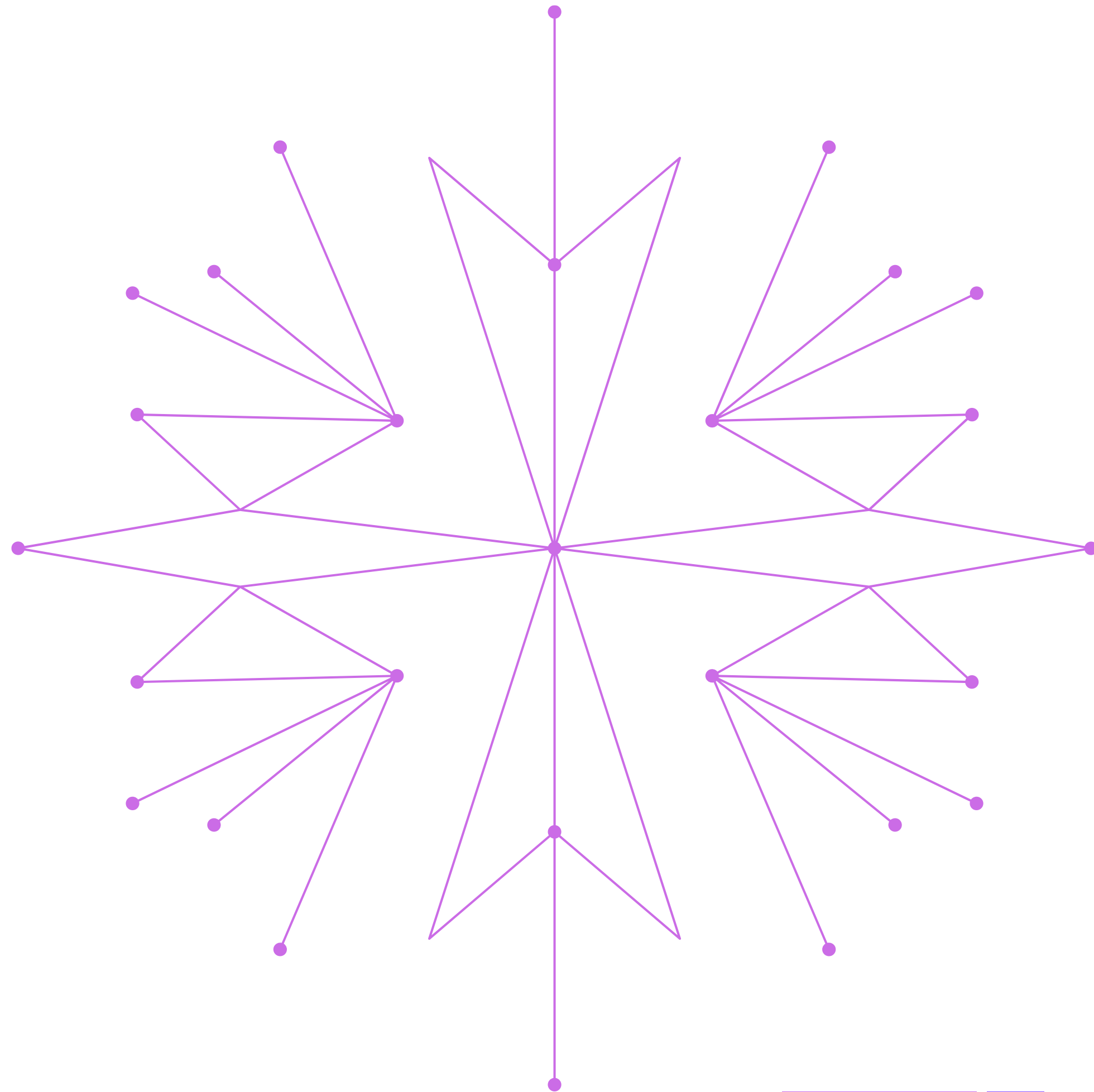
- Código-fonte
- Configuração do kernel
- Drivers e módulos
- Compilação
- Instalação
- Criação de módulos compiláveis
- Debugging
- Otimização

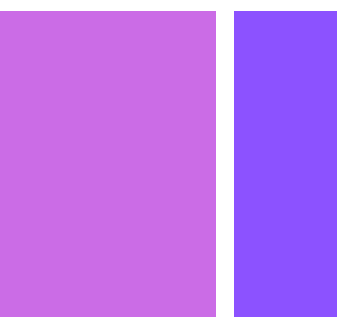




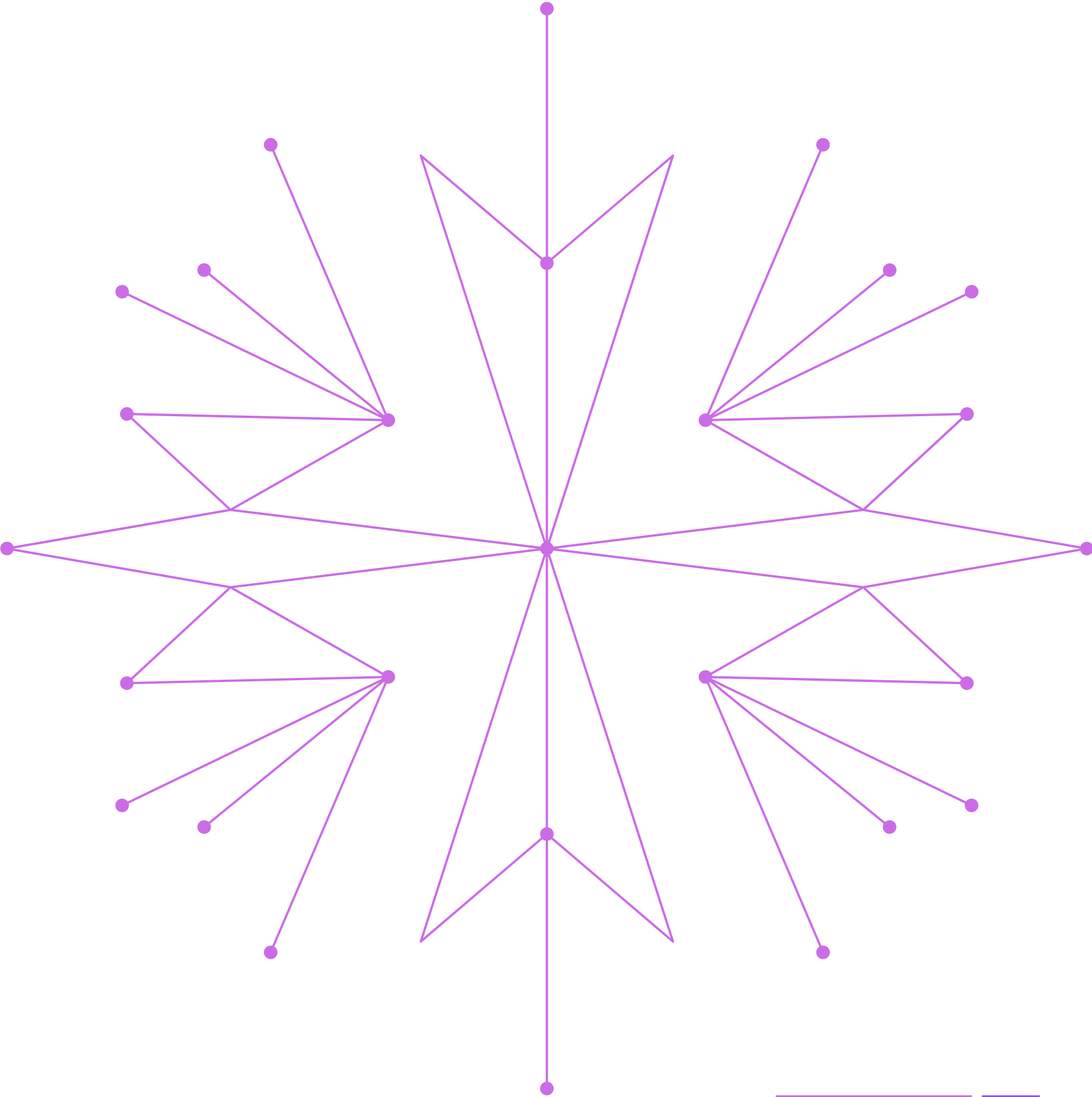
CÓDIGO-FONTE

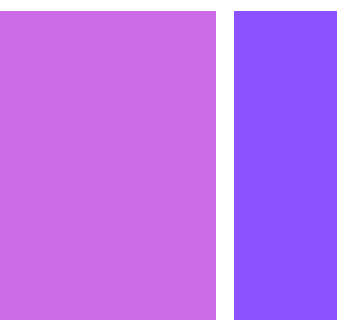
- Código-fonte = “Vanilla Kernel”
- Pureza (sem patches)
- Garantia de trabalhar sem dependências de distribuições específicas





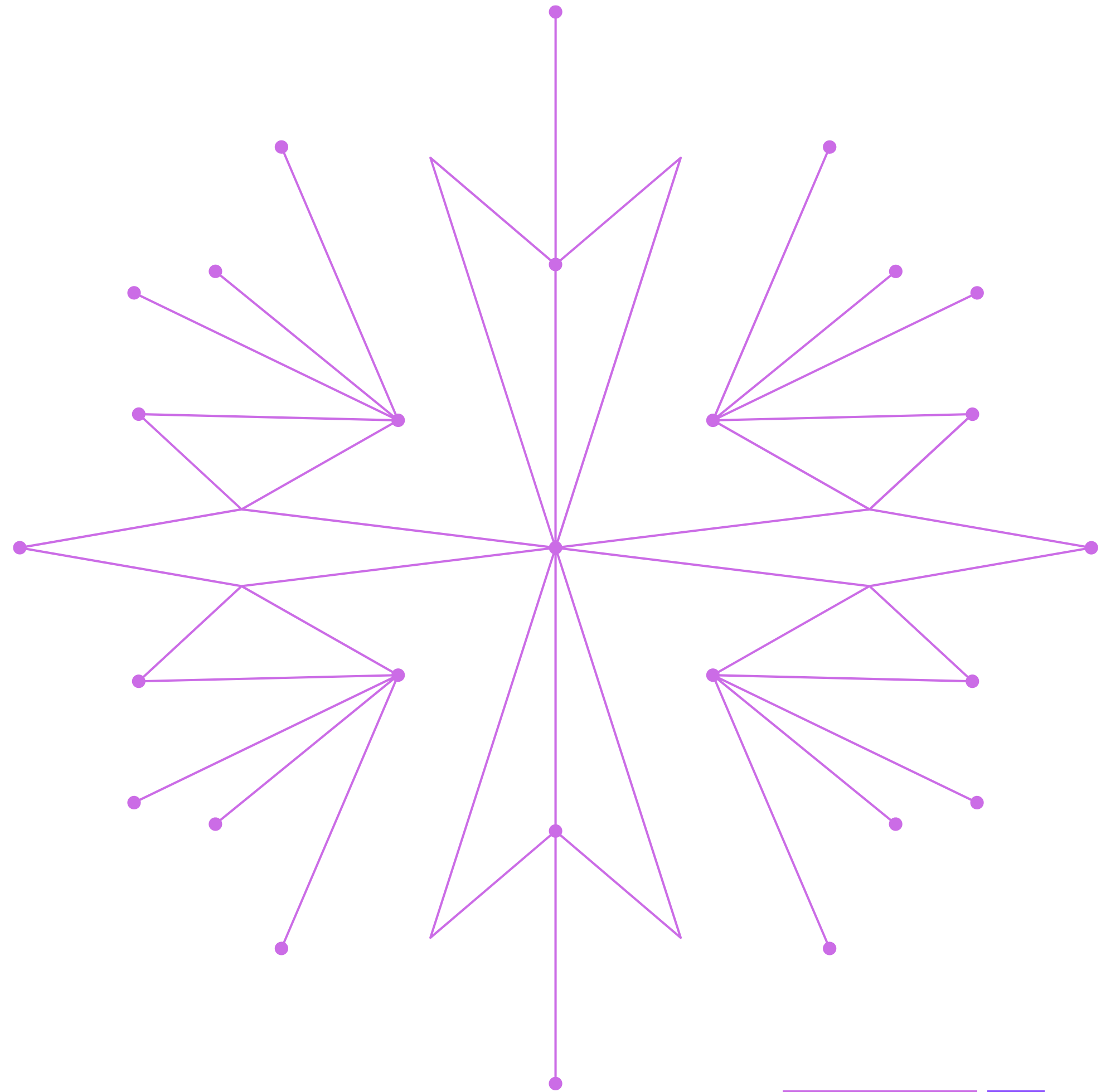
CÓDIGO-FONTE





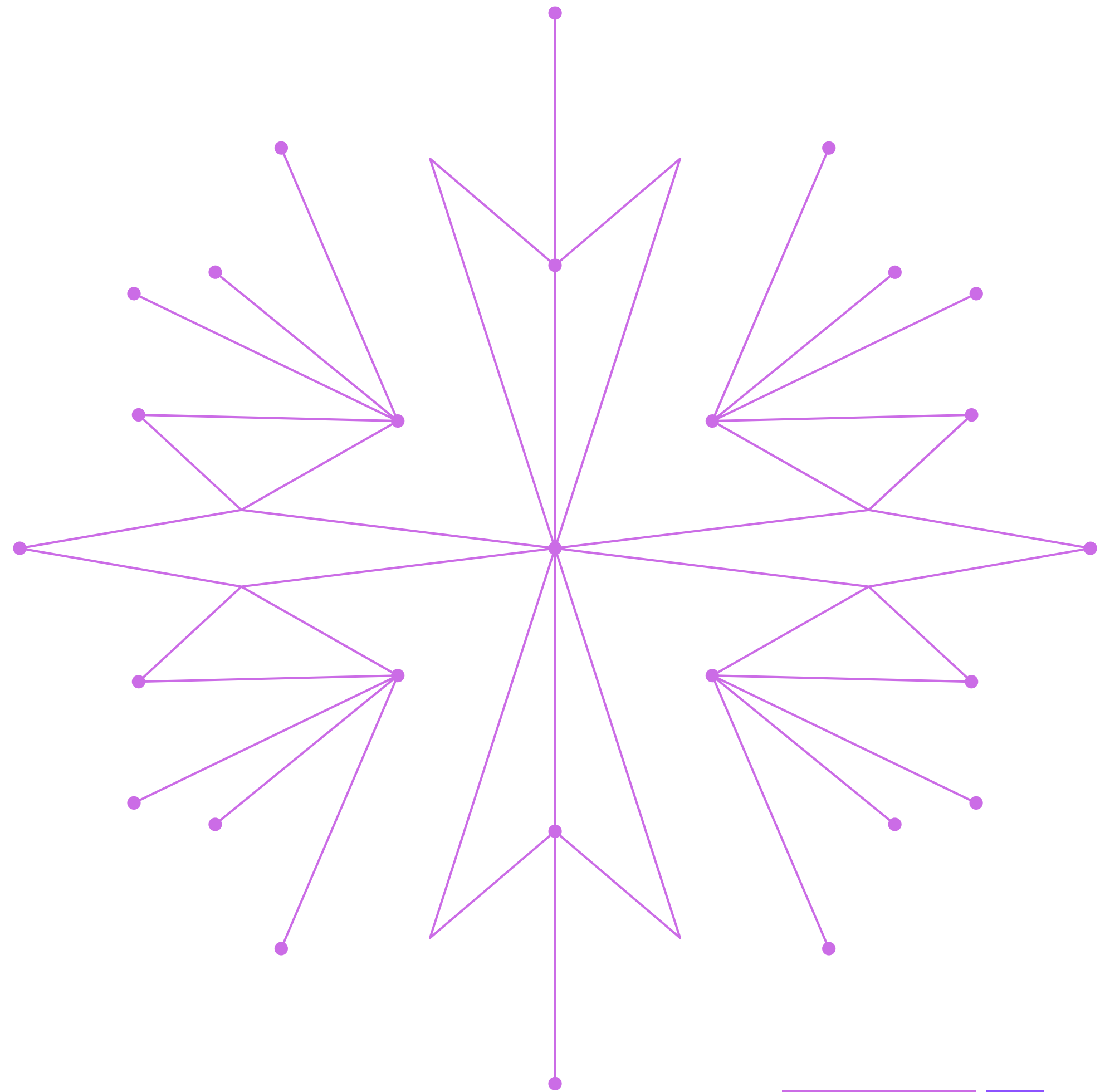
`ARCH/`

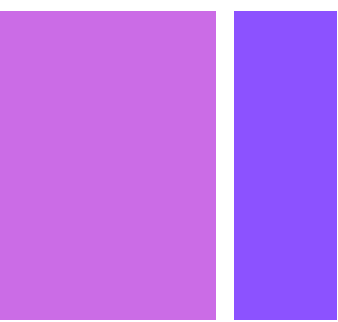
- Código que depende da arquitetura do hardware
- Há subdiretórios para cada arquitetura: ``x86/``, ``arm/``, ``arm64/``, ``mips/``, ``powerpc/``..
- Instruções específicas para lidar com a inicialização do processador e a transição para o modo protegido, a configuração das tabelas de paginação da MMU e o tratamento de interrupções de hardware (IRQs) através do Advanced Programmable Interrupt Controller (APIC)



`DRIVERS/`

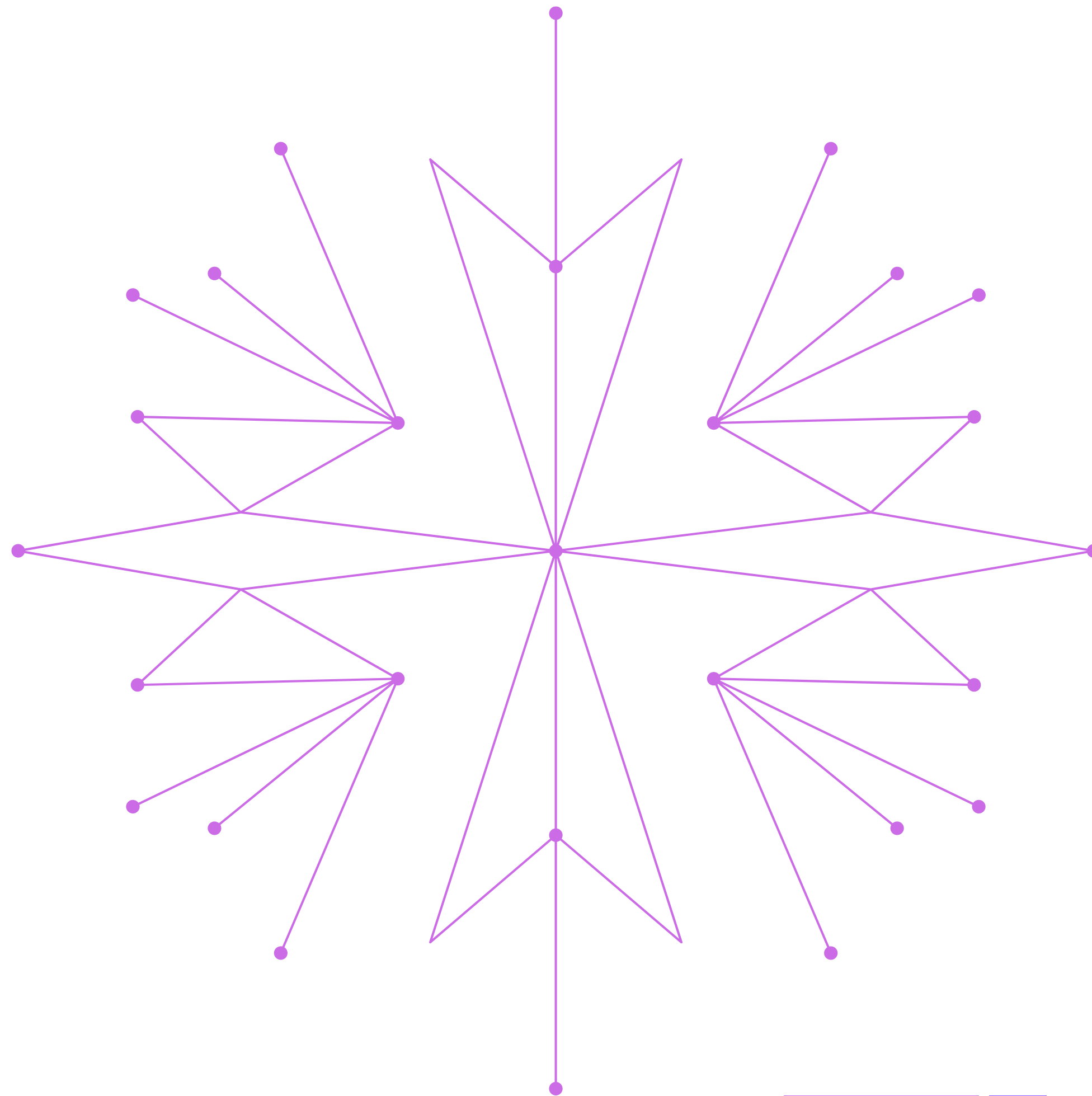
- Maior diretório (volume de código e número de arquivos)
- Device drivers, são os módulos de software responsáveis por instruir o kernel sobre como interagir e controlar o hardware físico
- Arquitetura monolítica
- `drivers/gpu/`
- `drivers/net/`
- `drivers/usb/`
- `drivers/block/` e `drivers/scsi`

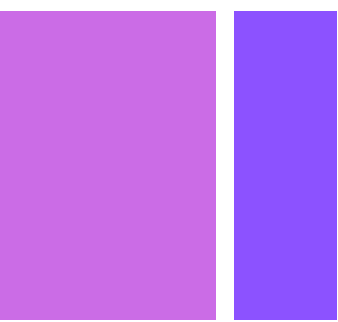




`FS/`

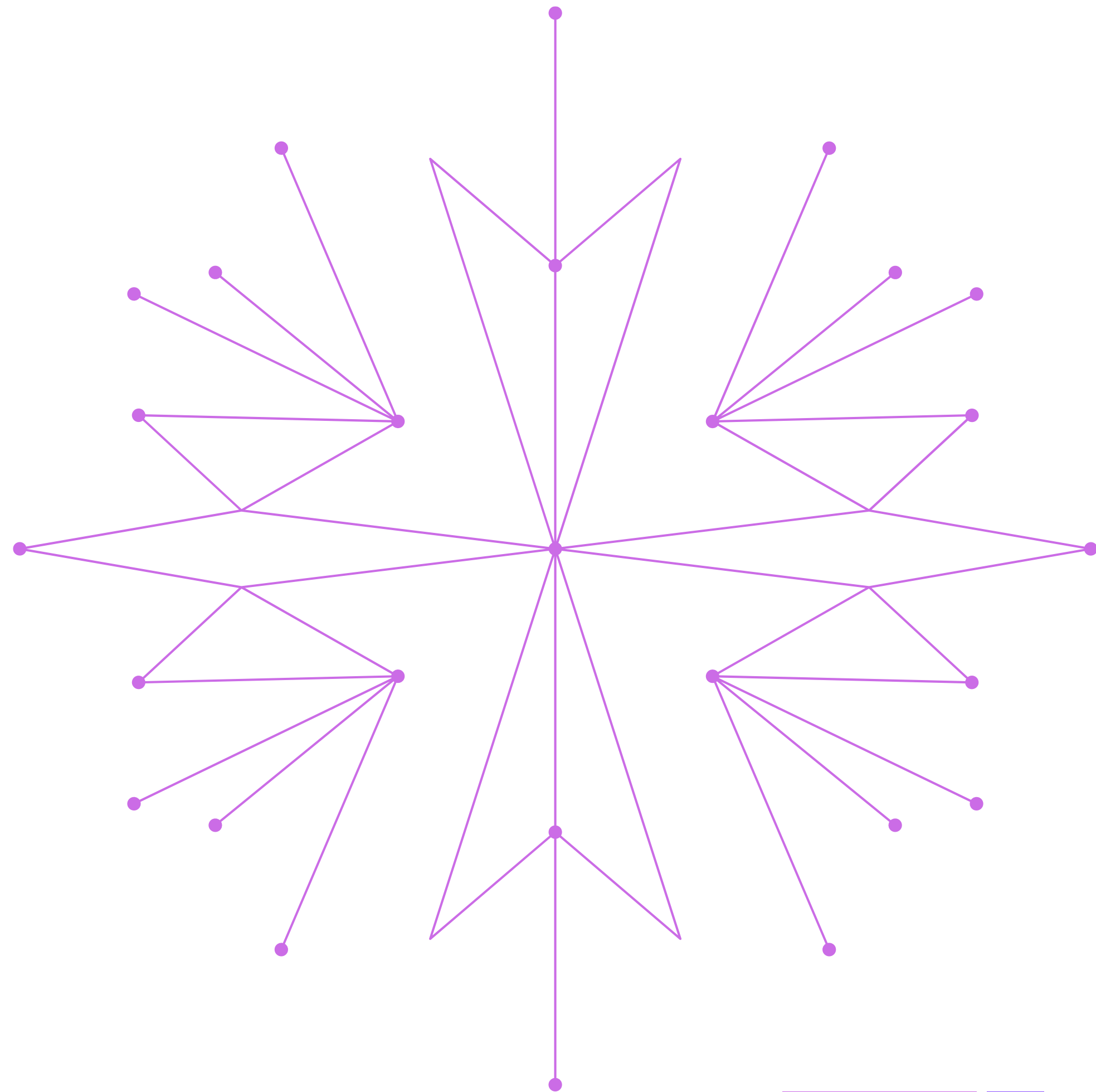
- Filesystems
- Implementações lógicas de sistemas de arquivos diferentes
- Virtual File System
- `fs/ext4/`
- `fs/btrfs/`
- `fs/fat/`
- `fs/proc/` e `fs/sysfs`

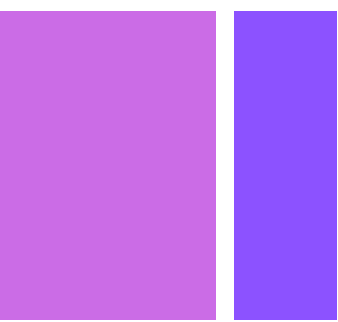




`KERNEL/`

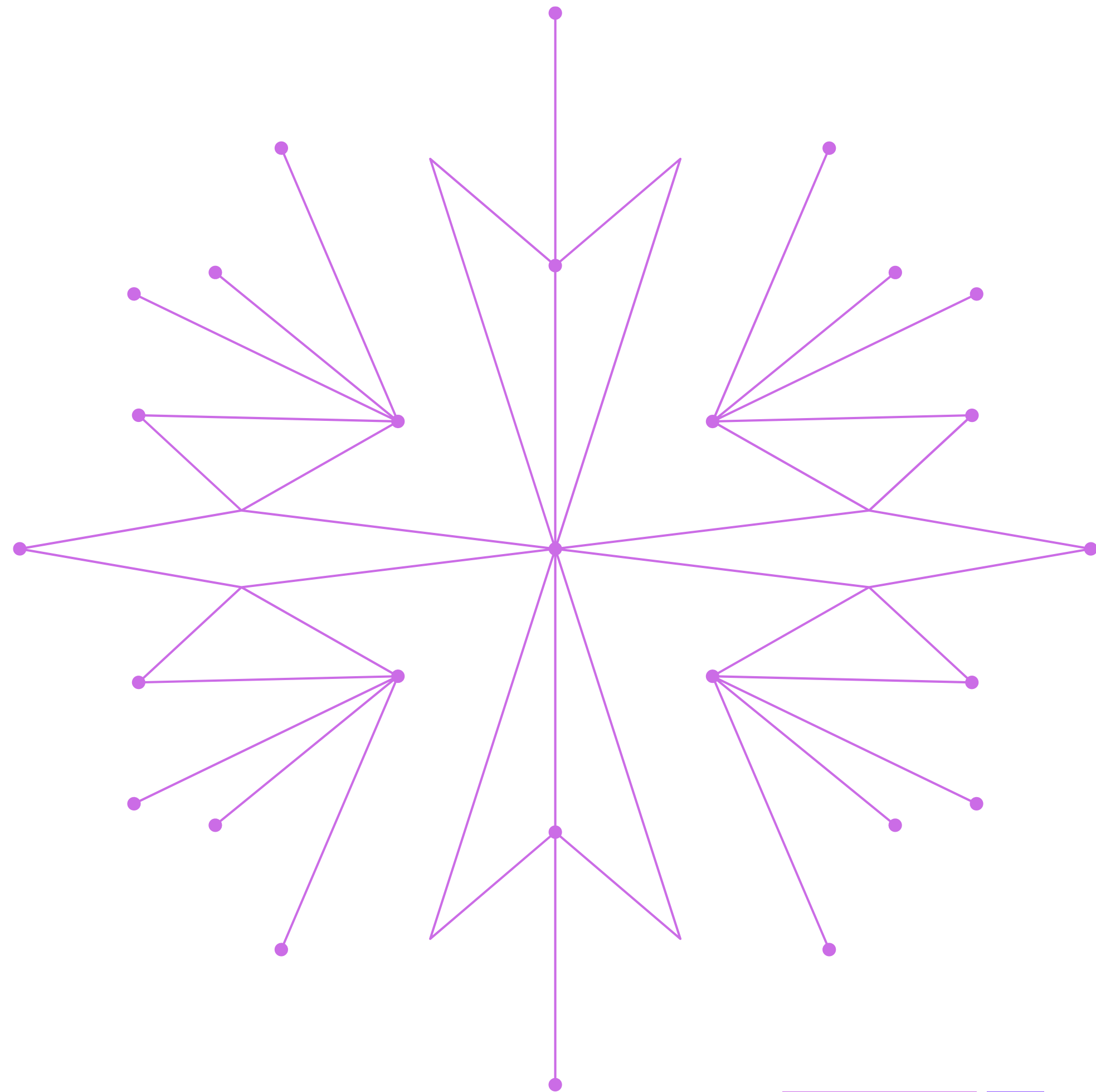
- Implementação dos mecanismos centrais que define um sistema operacional
- Agnóstico em relação à arquitetura de hardware
- Abstrações
- Scheduler
- Signals
- Timers
- Locking

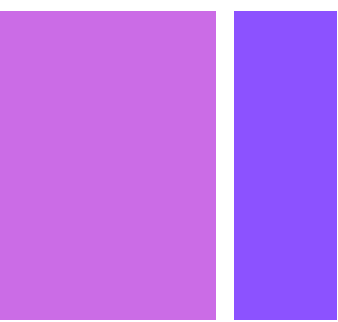




MM/

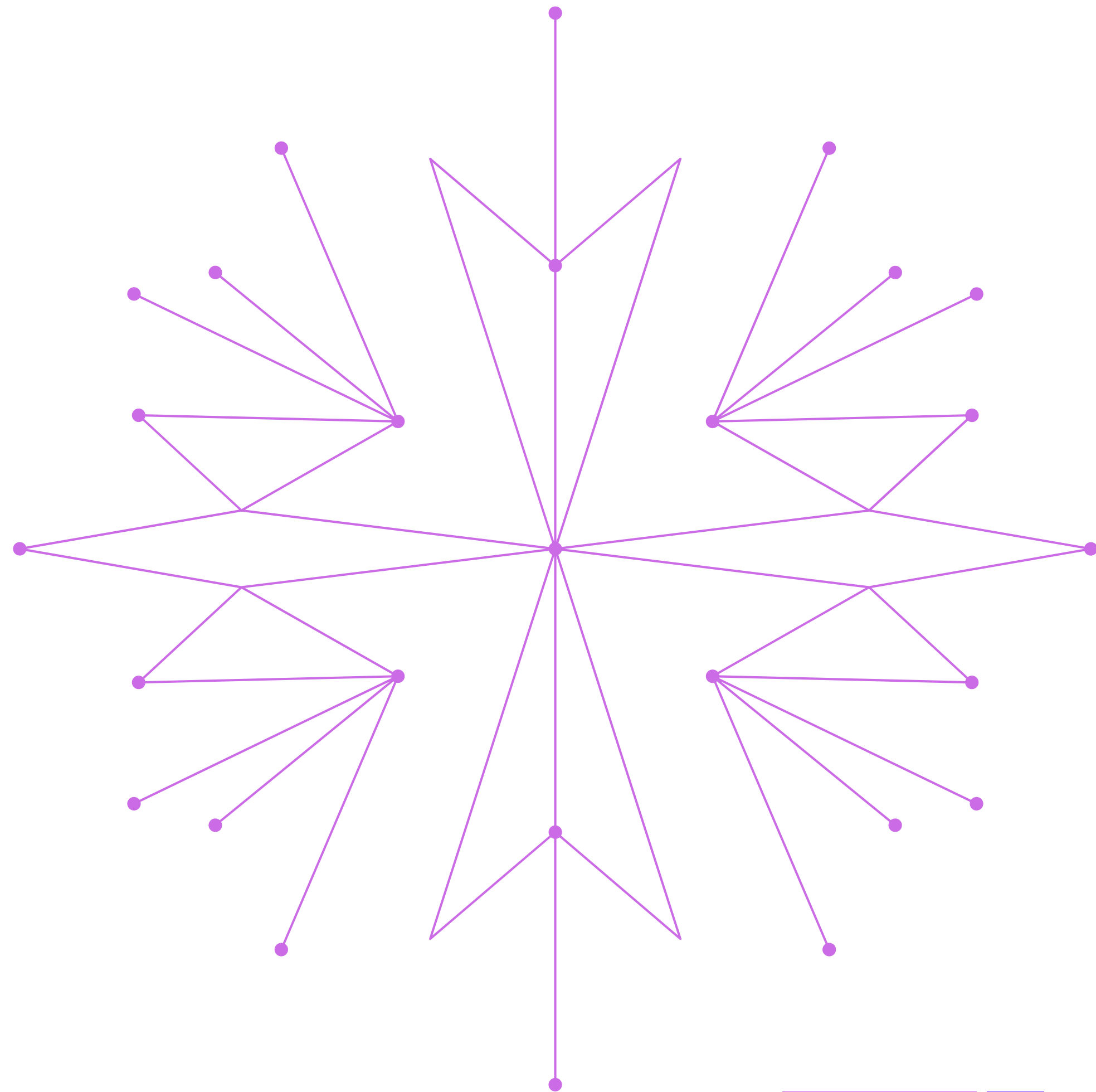
- Memory management (subsistema de gerenciamento de memória)
- Gerencia a alocação e liberação da memória RAM física e a implementação da memória virtual (gerencia a tradução dos endereços virtuais em endereços físicos através de tabelas de paginação)
- Demand Paging
- Swapping
- Page Cache

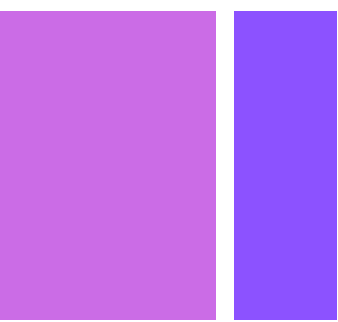




`NET/`

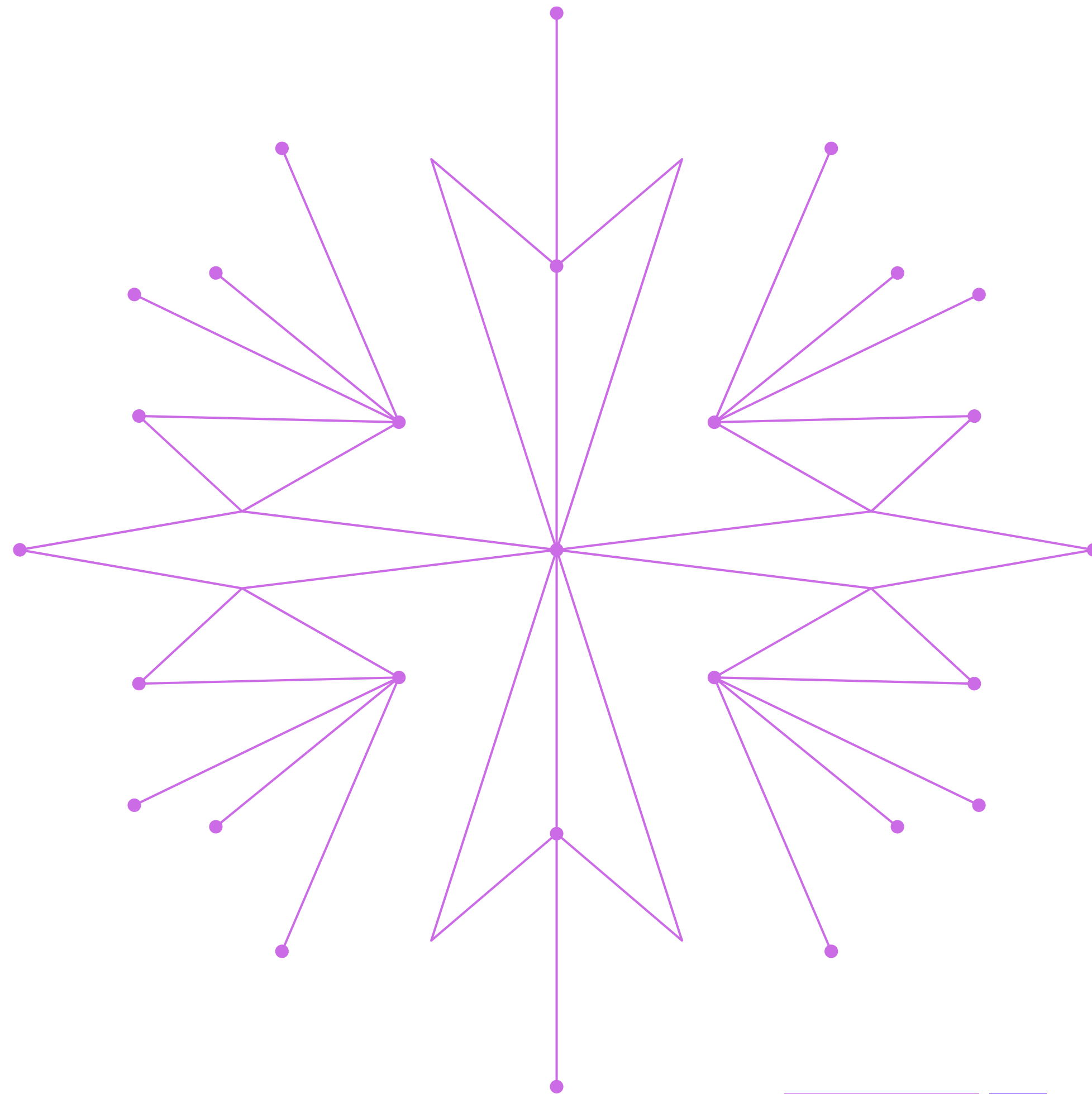
- Implementação dos protocolos de comunicação de rede (pilhas IPv4 e IPv6, controle de congestionamento e montagem de pacotes do protocolo TCP, transmissão datagrama do UDP e funcionalidades avançadas de roteamento, filtragem de pacotes e qualidade de serviço.
- `include/`
- `scripts/`
- `lib/`





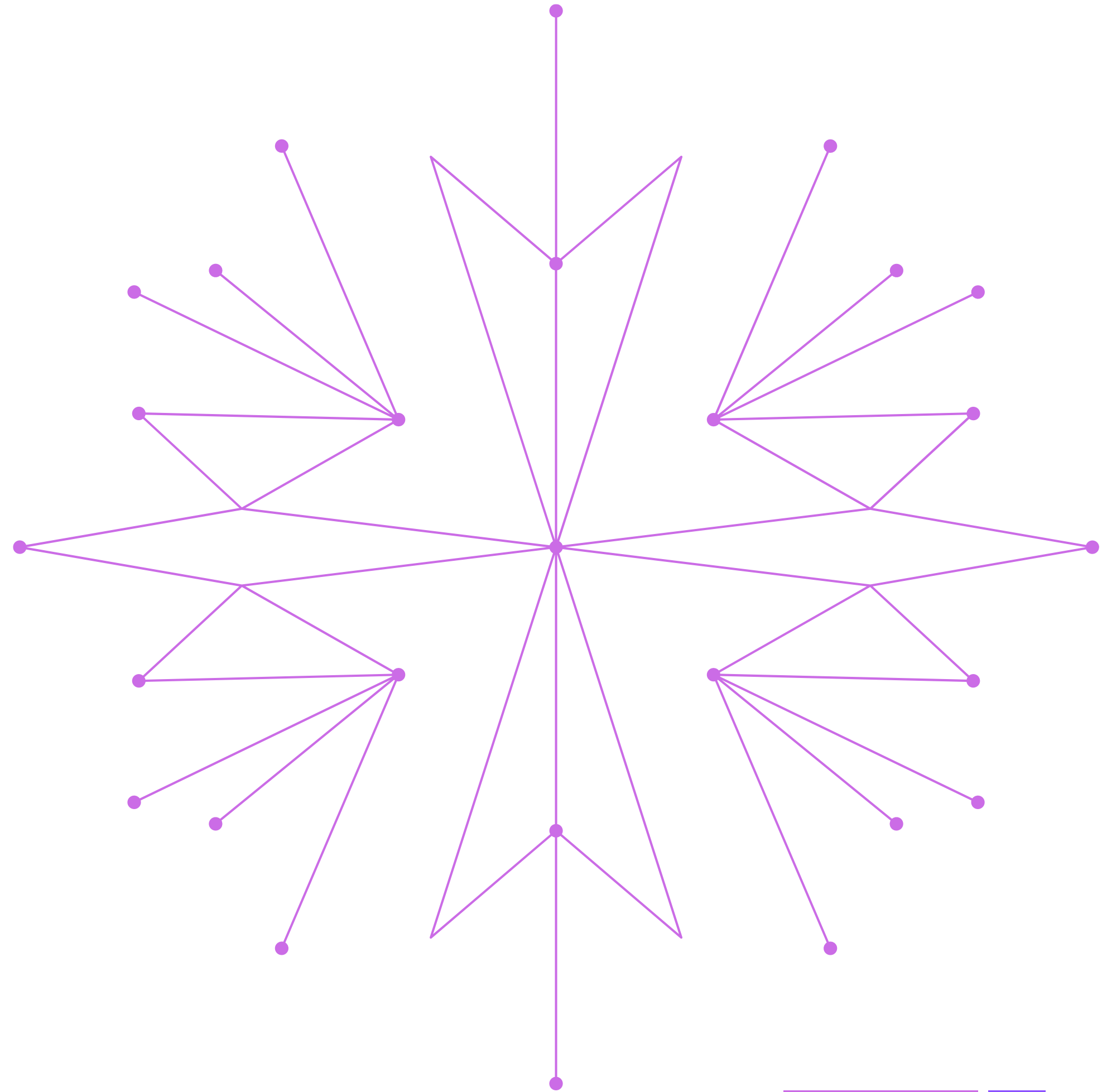
CONFIGURAÇÃO

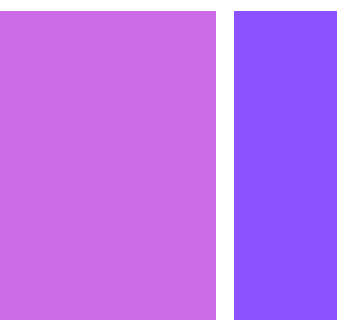
- Kbuild é o sistema de build do Linux
- Ele é responsável por interpretar a configuração do kernel, o que será compilado e coordenar o processo de geração do kernel e dos módulos
- ``.config`` = Manual de instruções para o compilador GCC e para o linker
- Uma lista de variáveis de ambiente e diretivas de configuração (Kconfig symbols)



A TRINDADE

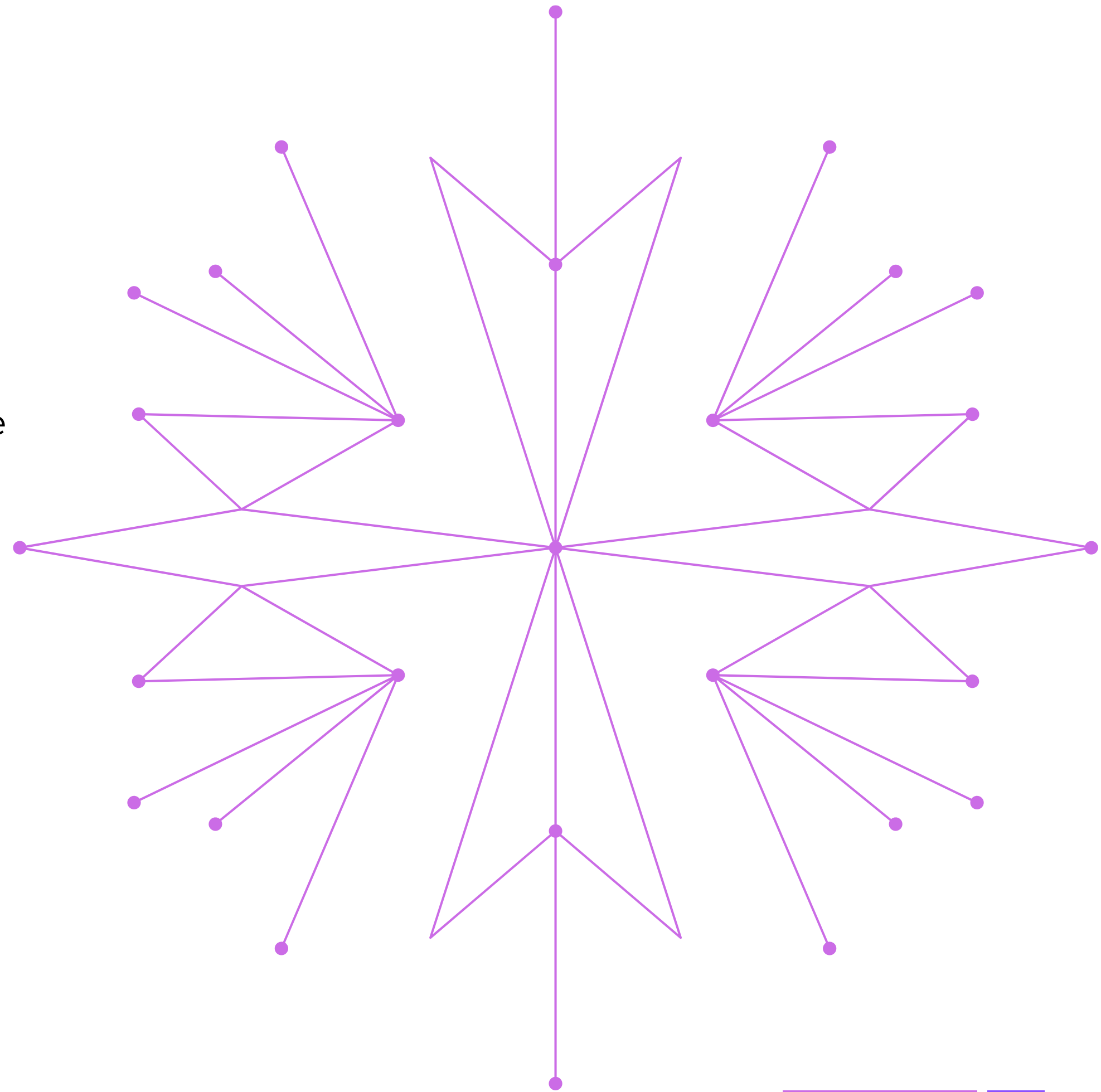
- Kbuild oferece um modelo de decisão tripla
- `y`
 - O sistema de build instrui o compilador a traduzir o código-fonte correspondente e instrui o linker a colar o arquivo objeto resultando (.o) dentro do binário executável final do kernel
- `m`
- `n`

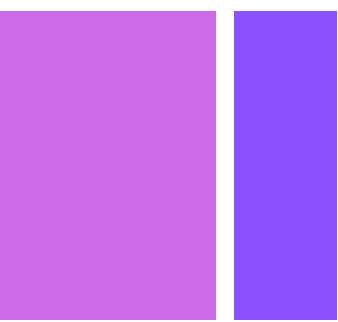




LFS

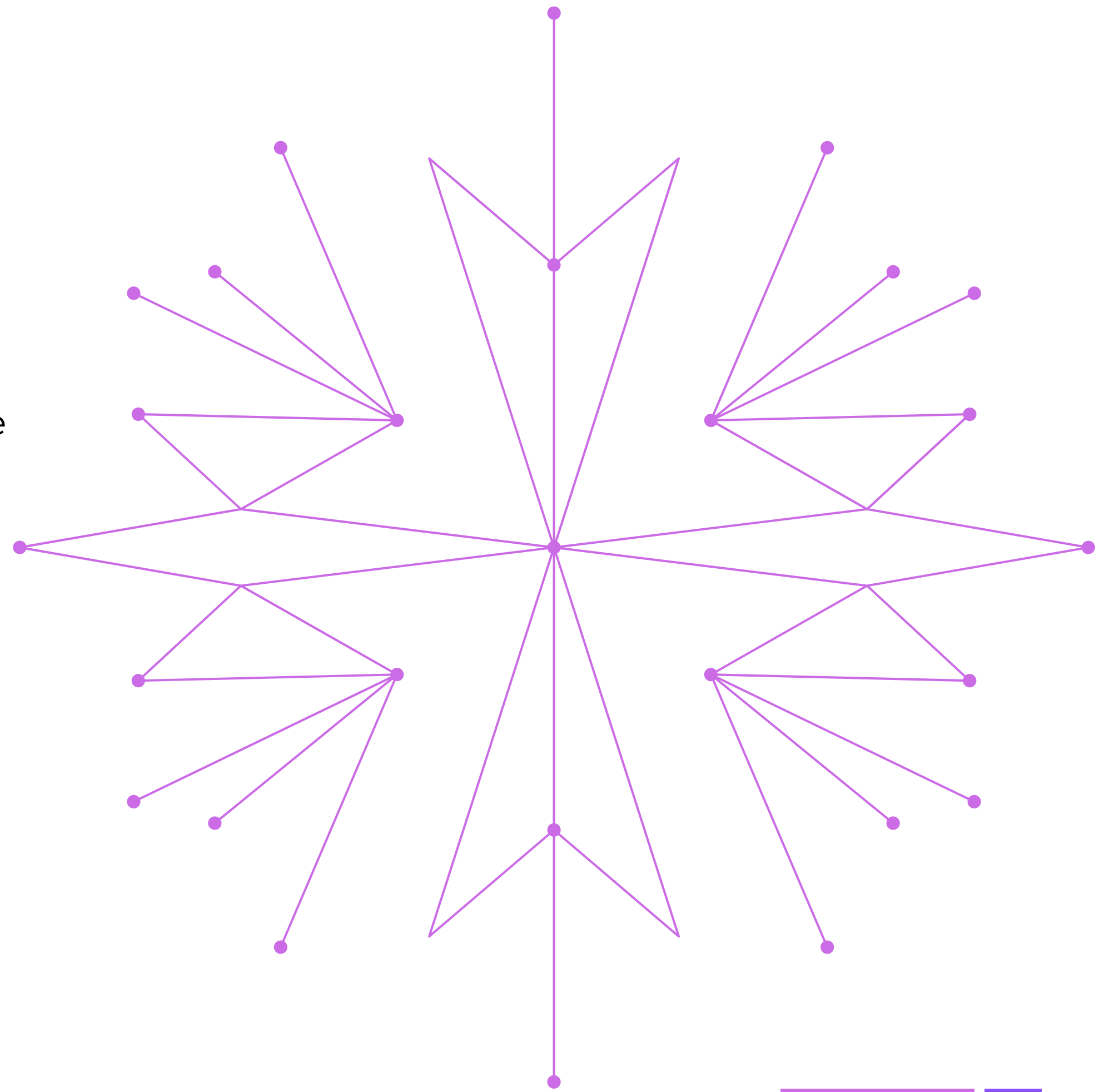
- Construir um sistema Linux a partir do código-fonte

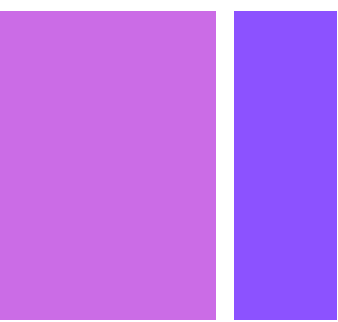




KERNEL

- Construir um sistema Linux a partir do código-fonte





KERNEL LINUX

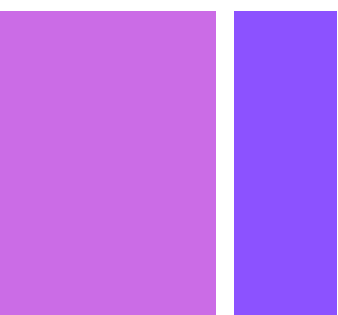
É a ponte entre o hardware e software (Sistema Operacional)

Núcleo do sistema operacional

Gerencia tanto o hardware quanto software

Responsável por:

- Memória
 - Processos
 - Segurança
 - Dispositivos
- 



CÓDIGO-FONTE DO KERNEL

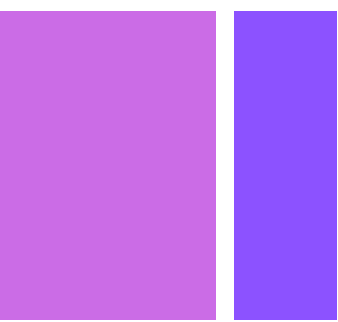
O Kernel do Linux possui código aberto

É feito em C

É mantido pela comunidade

```
(caio@kali)-[/  
$ cd usr/src  
  
(caio@kali)-[/usr/src  
$ ls  
linux-headers-6.18.5+kali-amd64  linux-kbuild-6.18.5+kali  
linux-headers-6.18.5+kali-common python3.13
```





CONFIGURAÇÃO DO KERNEL

Ferramentas de configuração:

make menuconfig

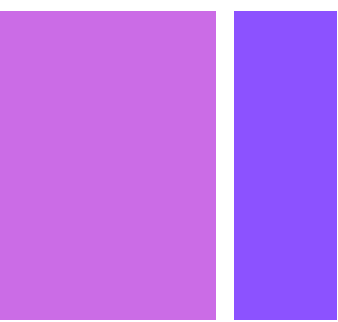
make xconfig

make oldconfig

Por meio delas é possível:

- Escolher funcionalidades
- Ativar/desativar drivers
- Definir módulos





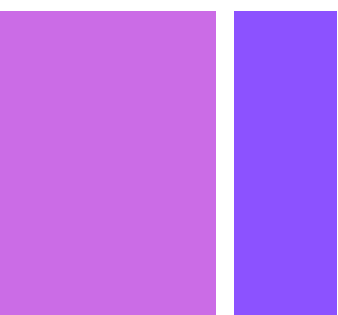
DRIVERS E MÓDULOS ESSENCIAIS

Drivers essenciais:

- Disco/SSD
- Sistema de arquivos
- Rede
- USB

Vantagens dos módulos:

- Menor consumo
 - Flexibilidade
 - Atualização sem recompilar tudo
- 



COMPILAÇÃO DO KERNEL

Compila o kernel:

make

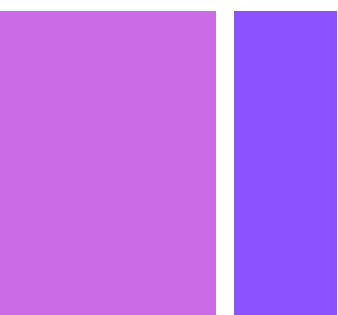
Compila os módulos:

make modules

Instala módulos:

make modules_install





INSTALAÇÃO DO KERNEL

Arquivos importantes:

 vmlinuz (Imagem compactada do kernel)

 System.map (Mapa de símbolos do kernel)

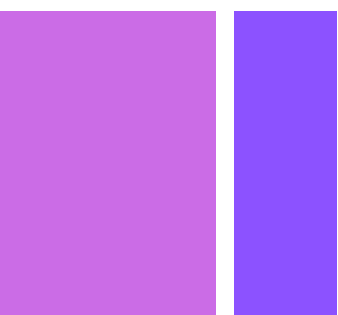
Diretório

 /boot

Atualização do boot-loader:

 upgrade-grub





MÓDULOS COMPILÁVEIS

Exemplo básico:

```
#include <linux/module.h>  
MODULE_LICENSE("GPL");
```

Compilação:

```
make -C /lib/modules/$(uname -r)/build M=$PWD modules
```

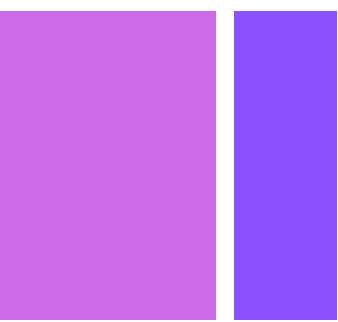
Carregamento:

```
insmod modulo.ko
```

Remoção:

```
rmmmod modulo
```





DEBUGGING DE PROBLEMAS

Problemas comuns:

- Dependências comuns

- Versão incompatível

- Erros de sintaxe

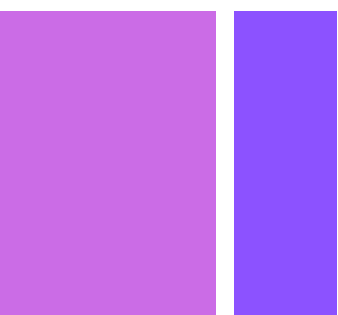
- Falta de drivers ou drivers corrompidos

Ferramentas que possam ser úteis:

- journalctl***

- dmesg***





OTIMIZAÇÃO DO KERNEL

Remover módulos desnecessários

Compilar somente drivers que serão utilizados

Ajustar escalonamento

Otimizar consumo de memória

Esse conjunto melhora o desempenho do Kernel, fazendo uma inicialização mais rápida e um menor uso de recursos

